ATS ADMINISTRASI JARINGAN

NAMA : VIANI DESTIANA PAMBUDI
KELAS : XII TKJ 2
ABSEN : 35

Web kamu dibuat pakai apa?

Pada project utsmicroservice, saya menggunakan beberapa teknologi utama yaitu:

1. **Python**
   Bahasa utama yang digunakan dalam project ini adalah **Python**.
   Python digunakan untuk membuat **backend** dan **API** yang bertugas untuk mengelola data serta menghubungkan sistem dengan database dan storage.

   Fungsi python dalam project ini adalah :
   - Membuat backend service
   - Membuat API service
   - Menghubungkan aplikasi dengan database PostgreSQL
   - Mengatur upload file foto ke MinIO

   Dengan Python, sistem bisa memproses data dari user dan menyimpannya ke database.

2. **HTML dan Javascript**
   Selain Python, project ini juga menggunakan HTML dan Javascript untuk bagian **frontend**.
   Frontend adalah tampilan web yang dilihat oleh pengguna**.**

   HTML
   HTML digunakan untuk membuat struktur halaman web, seperti:
   - Membuat halaman website
   - Membuat form input data user
   - Menampilkan daftar user
   - membuat tombol upload foto

   Javascript
   Javascript digunakan untuk membuat halaman web menjadi **interaktif**.
   Fungsinya adalah:
   - Mengirim request dari web ke backend
   - Mengambil data user dari API

- menampilkan data user ke halaman web

Dengan Javascript, web bisa berkomunikasi dengan server tanpa harus reload halaman.

3. **Framework Python**

   Dalam project ini saya menggunakan dua framework Python, yaitu **Flask dan FastAPI**.

   Flask

   Flask digunakan pada folder **backend**. Flask adalah framework Python yang digunakan untuk membuat **web server sederhana**.

   Fungsi Flask dalam project ini adalah:
   - Menerima request dari frontend
   - Meneruskan request tersebut ke API
   - Menjadi perantara antara frontend dan API

   Jadi Flask berperan sebagai gateway atau penghubung antara tampilan web dengan API.

   Alur kerjanya:

   Frontend → Backend (Flask) → API (FastAPI)

   FastAPI

   FastAPI digunakan pada folder **api**.
   FastAPI adalah framework Python yang digunakan untuk membuat REST API dengan performa tinggi**.**
   FastAPI berfungsi sebagai **service utama yang mengelola data**.

   Fungsi FastAPI dalam project ini adalah:
   - Membuat endpoint API seperti:
     - GET user
     - POST user
     - DELETE user
     - UPDATE user
   - Melakukan proses **CRUD (Create, Read, Update, Delete)**
   - Menyimpan data user ke **database PostgreSQL**
   - Mengupload file foto user ke **MinIO**

   Karena semua proses utama dilakukan di sini, maka **FastAPI dapat disebut sebagai otak utama sistem**.

4. **Storage File**

   Project saya ini menggunakan **MinIO**.
   MinIO adalah **object storage** seperti Google Drive atau AWS S3.

Fungsinya:

- Menyimpan file foto user
- Supaya database tidak menyimpan file besar

5. **Docker**
   Project kamu dijalankan menggunakan **Docker**.
   Docker berfungsi untuk:
   - Menjalankan semua aplikasi dalam container
   - Membuat environment yang sama di semua komputer

KESIMPULAN

Jadi, web ini dibuat menggunakan bahasa pemrograman Python dengan framework Flask untuk backend dan FastAPI untuk API service.
Frontend menggunakan HTML dan Javascript.
Data disimpan di database PostgreSQL dan file foto disimpan di MinIO. Semua service dijalankan menggunakan Docker dan docker-compose sehingga sistem dapat berjalan dalam container secara terpisah.

STRUKTUR PROJECT

```
utsmicroservice/
│
├── frontend/
│   ├── index.html
│   └── Dockerfile
│
├── backend/
│   ├── main.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── api/
│   ├── main.py
│   ├── database.py
│   ├── models.py
│   ├── requirements.txt
│   └── Dockerfile
│
├── docker-compose.yml
└── .env
```

1. **Folder Frontend**
   frontend/
   │

```
├── index.html
└── Dockerfile
```

Frontend adalah **bagian yang akan dilihat oleh user**.

- **Index.html**
  Ini adalah **halaman web utama**.

  Fungsinya:
  - Menampilkan daftar user
  - Menampilkan form input
  - Tombol tambah user
  - Tombol delete user
  - Upload foto

  Contoh alurnya:
  User buka web
  ↓
  User isi form
  ↓
  Javascript kirim data ke backend

  Berikut merupakan isi dari index.html:

```
  GNU nano 8.4                                                    index.html
<!DOCTYPE html>
<html>
<head>
    <title>UTS ASJ</title>
</head>
<body>

<h2>Tambah User</h2>

<form action="http://10.63.115.217:8000/users" method="post" enctype="multipart/form-data">
    Nama: <input type="text" name="name"><br><br>
    Email: <input type="text" name="email"><br><br>
    Foto: <input type="file" name="photo"><br><br>
    <button type="submit">Tambah</button>
</form>

<hr>

<button onclick="loadUsers()">Lihat Users</button>
<ul id="list"></ul>

<script>
async function loadUsers(){
    const res = await fetch("http://localhost:5000/users");
    const data = await res.json();

    let list = document.getElementById("list");
    list.innerHTML = "";

    data.forEach(user=>{
        list.innerHTML += `<li>${user.name} - ${user.email}</li>`;
                                                          [ Read 37 lines ]
^G Help        ^O Write Out   ^F Where Is    ^K Cut       ^T Execute     ^C Location    M-U Undo       M-A S
^X Exit        ^R Read File   ^\ Replace     ^U Paste     ^J Justify     ^/ Go To Line  M-E Redo       M-6 C
```

- **Dockerfile**
  File ini digunakan oleh Docker untuk membuat **container frontend**.

Docker akan menjalankan web saya menggunakan web server seperti **Nginx**.
Berikut merupakan isi dari Dockerfile:

```
  GNU nano 8.4                                                    Dockerfile
FROM nginx:alpine
COPY . /usr/share/nginx/html
```

2. **Folder Backend**
   backend/
   ```
   │
   ├── main.py
   ├── requirements.txt
   └── Dockerfile
   ```
   Backend ini menggunakan **Python Flask**.
   Fungsinya sebagai **perantara antara frontend dan API**.

   - **main.py**
     Ini adalah **server backend**.

     Fungsinya:
     - menerima request dari frontend
     - meneruskan request ke API FastAPI

     Contoh request:
     GET /users
     POST /users
     DELETE /users

     Jadi alurnya:
     Frontend
     ↓
     Backend (Flask)
     ↓
     API (FastAPI)

     Berikut merupakan isi dari main.py:

```
  GNU nano 8.4                                                    main.py
from flask import Flask, jsonify
import requests

app = Flask(__name__)

API_URL = "http://api:8000"

@app.route("/")
def home():
    return "Backend Running"

@app.route("/users")
def users():
    res = requests.get(f"{API_URL}/users")
    return jsonify(res.json())

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

- **requirements.txt**
  File ini berisi **library Python yang dibutuhkan backend**.
  Saat Docker build container, semua library ini akan diinstall.

  Berikut merupakan isi dari requirements.txt:

```
  GNU nano 8.4                                            requirements.txt
flask
requests
```

- **Dockerfile**
  File ini digunakan untuk membuat **container backend**.
  Docker akan menjalankan aplikasi Flask dari sini.

  Berikut merupakan isi dari Dockerfile:

```
  GNU nano 8.4                                                Dockerfile
FROM python:3.10

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

3. **Folder API**

api/

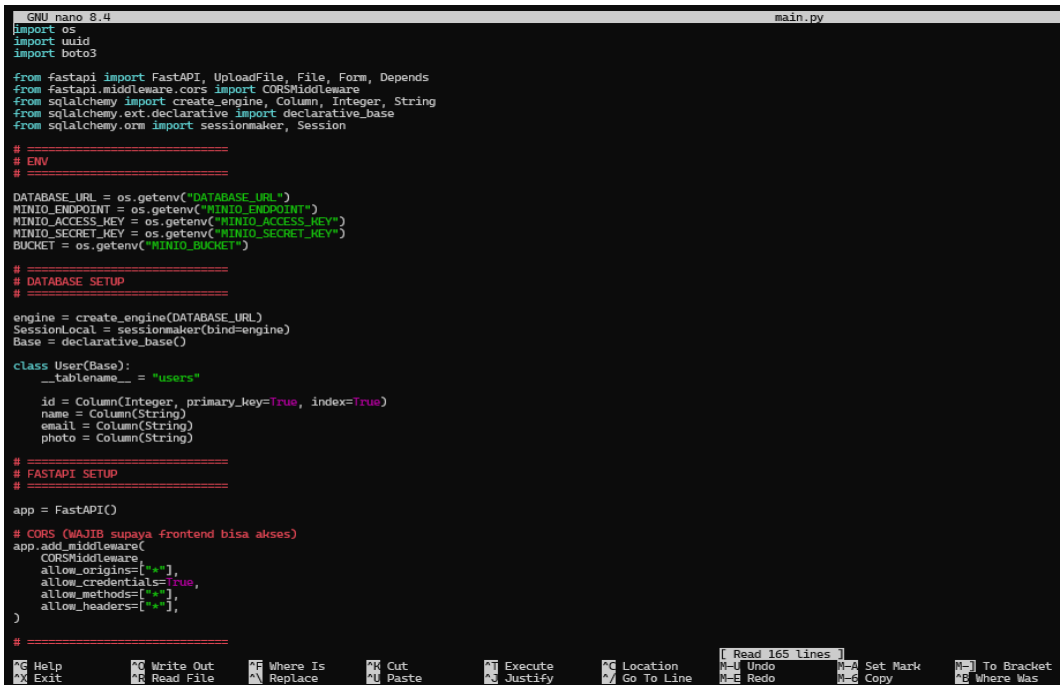
```
│
├── main.py
├── database.py
├── models.py
├── requirements.txt
└── Dockerfile
```

Ini adalah bagian **service utama**.
API ini menggunakan **FastAPI**.

- **main.py**
  Ini adalah **logic utama API**.
  Berisi endpoint seperti:
  GET /users
  POST /users
  PUT /users/{id}
  DELETE /users/{id}

  Fungsinya:
  - Mengambil data user
  - Menambah user
  - Menghapus user
  - Mengubah user
  - Upload foto ke MinIO

  Berikut merupakan isi dari main.py



```
  GNU nano 8.4                                                    main.py
import os
import uuid
import boto3

from fastapi import FastAPI, UploadFile, File, Form, Depends
from fastapi.middleware.cors import CORSMiddleware
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, Session

# ============================
# ENV
# ============================

DATABASE_URL = os.getenv("DATABASE_URL")
MINIO_ENDPOINT = os.getenv("MINIO_ENDPOINT")
MINIO_ACCESS_KEY = os.getenv("MINIO_ACCESS_KEY")
MINIO_SECRET_KEY = os.getenv("MINIO_SECRET_KEY")
BUCKET = os.getenv("MINIO_BUCKET")

# ============================
# DATABASE SETUP
# ============================

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    email = Column(String)
    photo = Column(String)

# ============================
# FASTAPI SETUP
# ============================

app = FastAPI()

# CORS (WAJIB supaya frontend bisa akses)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# ============================
                                                  [ Read 165 lines ]
^G Help      ^O Write Out   ^F Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo      M-6 Copy       ^B Where Was
```

- **database.py**
  File ini digunakan untuk **koneksi ke database PostgreSQL**.
  Fungsinya untuk menghubungkan FastAPI dengan database.

  Berikut mesupakan isi dari database.py:

```
GNU nano 8.4                                                    database.py
import os
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

DATABASE_URL = os.getenv("DATABASE_URL")

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(bind=engine)
Base = declarative_base()
```

- **models.py**
  File ini berisi **struktur tabel database**.

  Contohnya:
  User
  id
  name
  email
  photo

  FastAPI menggunakan **SQLAlchemy** untuk membuat tabel berdasarkan model ini.

  Berikut merupakan isi dari models.py:

```
GNU nano 8.4                                                    models.py
from sqlalchemy import Column, Integer, String
from database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    email = Column(String)
    photo = Column(String)
```

- **requirements.txt**
  Berisi library yang dibutuhkan API.
  Library ini dipakai untuk:
  - Menjalankan API
  - Koneksi database
  - Upload file ke MinIO

  Berikut merupakan isi dari requirements.txt:

```
  GNU nano 8.4                                                    requirements.txt
fastapi
uvicorn
sqlalchemy
psycopg2-binary
python-multipart
boto3
```

- **Dockerfile**
  Digunakan untuk membuat **container FastAPI**.

  Berikut merupakan isi dari Dockerfile:

```
  GNU nano 8.4                                                        Dockerfile
FROM python:3.10

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "main.py"]
```

4. **Docker-compose.yml**
   Ini adalah file yang **menjalankan semua container sekaligus**.
   Di dalam file ini biasanya ada service seperti:
   frontend
   backend
   api
   postgres
   minio

   Fungsi docker-compose:
   - Menjalankan banyak container sekaligus
   - Menghubungkan container satu sama lain

   Contoh menjalankan project:

   docker compose up –build

```
debian@debian: ~
root@debian:~/utsmicroservice# docker compose down -v
[+] down 6/6
 ✔Container utsmicroservice-frontend-1 Removed                0.2ss
 ✔Container utsmicroservice-backend-1  Removed                10.2s
 ✔Container utsmicroservice-api-1      Removed                0.5s
 ✔Container utsmicroservice-minio-1    Removed                0.3s
 ✔Container utsmicroservice-db-1       Removed                0.3s
 ✔Network utsmicroservice_default      Removed                0.1s
root@debian:~/utsmicroservice# nano docker-compose.yml
root@debian:~/utsmicroservice# docker compose up --build
[+] Building 16.7s (28/28) FINISHED
 => [internal] load local bake definitions                     0.0s
 => => reading from stdin 1.40kB                               0.0s
 => [frontend internal] load build definition from Dockerfile  0.0s
 => => transferring dockerfile: 84B                            0.0s
 => [backend internal] load build definition from Dockerfile   0.0s
 => => transferring dockerfile: 183B                           0.0s
 => [api internal] load build definition from Dockerfile       0.0s
 => => transferring dockerfile: 224B                           0.0s
 => [frontend internal] load metadata for docker.io/library/nginx  0.0s
 => [frontend internal] load .dockerignore                     0.0s
 => => transferring context: 2B                                0.0s
 => [backend internal] load metadata for docker.io/library/pytho  16.2s
 => [frontend internal] load build context                     0.0s
 => => transferring context: 61B                               0.0s
 => [frontend 1/2] FROM docker.io/library/nginx:alpine         0.0s
 => CACHED [frontend 2/2] COPY . /usr/share/nginx/html         0.0s
 => [frontend] exporting to image                              0.0s
 => => exporting layers                                        0.0s
 => => writing image sha256:5c333684f5b26c05ea1e7a4c5d53333172a33  0.0s
 => => naming to docker.io/library/utsmicroservice-frontend    0.0s
 => [frontend] resolving provenance for metadata file          0.0s
 => [api internal] load .dockerignore                          0.0s
 => => transferring context: 2B                                0.0s
 => [backend internal] load .dockerignore                      0.0s
```

Maka semua service langsung berjalan.

Berikut merupakan isi dari docker-compose.yml:

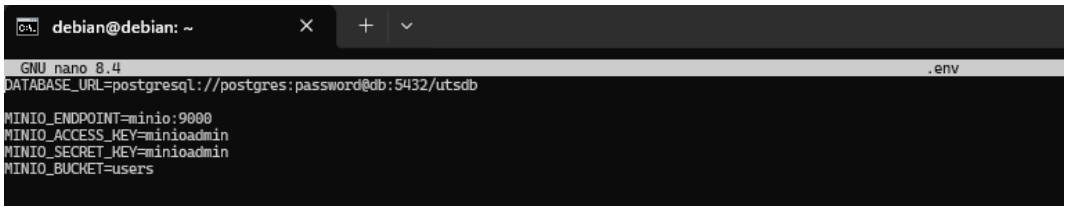
```
debian@debian: ~
  GNU nano 8.4                                   docker-compose.yml
services:

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: utsdb
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5


  minio:
    image: minio/minio
    command: server /data --console-address ":9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    ports:
      - "9000:9000"
      - "9001:9001"

  api:
    build: ./api
    ports:
      - "8000:8000"
    depends_on:
      db:
        condition: service_healthy
      minio:
        condition: service_started
    env_file:
      - .env

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - api

  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    depends_on:
      - backend

^G Help      ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-B Previous  ◂ Back
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^B Where Was    M-F Next      ▸ Forwar
```

5. **File .env**
   File **.env** digunakan untuk menyimpan **konfigurasi environment**.

   Fungsinya:
   - Menyimpan konfigurasi

- Supaya tidak ditulis langsung di kode
- Lebih aman

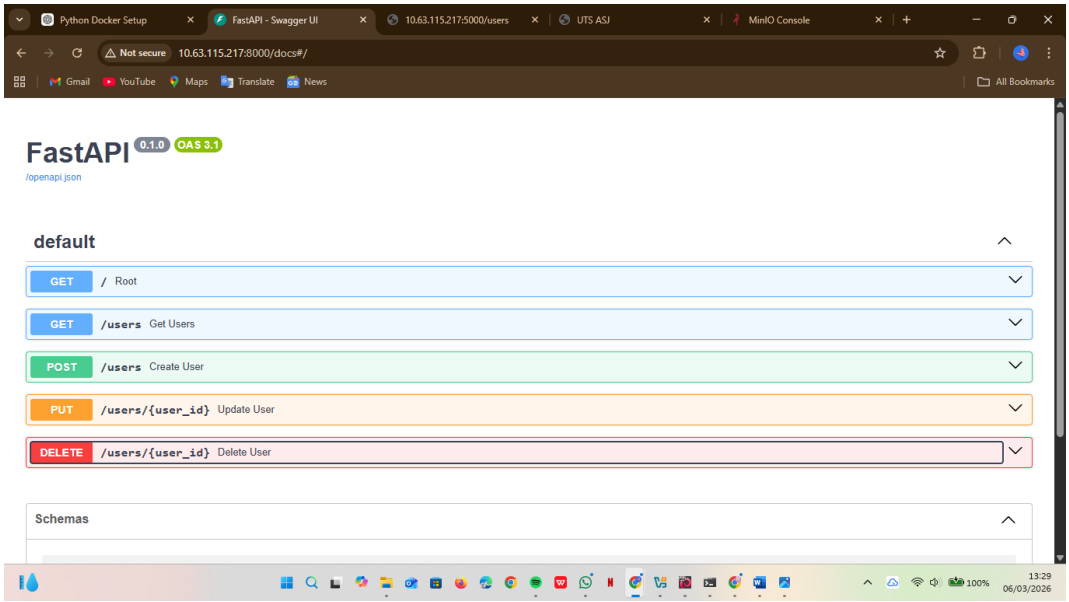
Berikut merupakan isi dari .env:



Setelah seluruh container berhasil dijalankan menggunakan Docker Compose, tahap selanjutnya adalah melakukan pengujian sistem melalui web browser untuk memastikan setiap layanan berjalan dengan baik. Pengujian dilakukan dengan mengakses beberapa alamat URL yang telah disediakan oleh masing-masing service. Berikut merupakan alamat yang digunakan untuk melakukan pengecekan sistem:

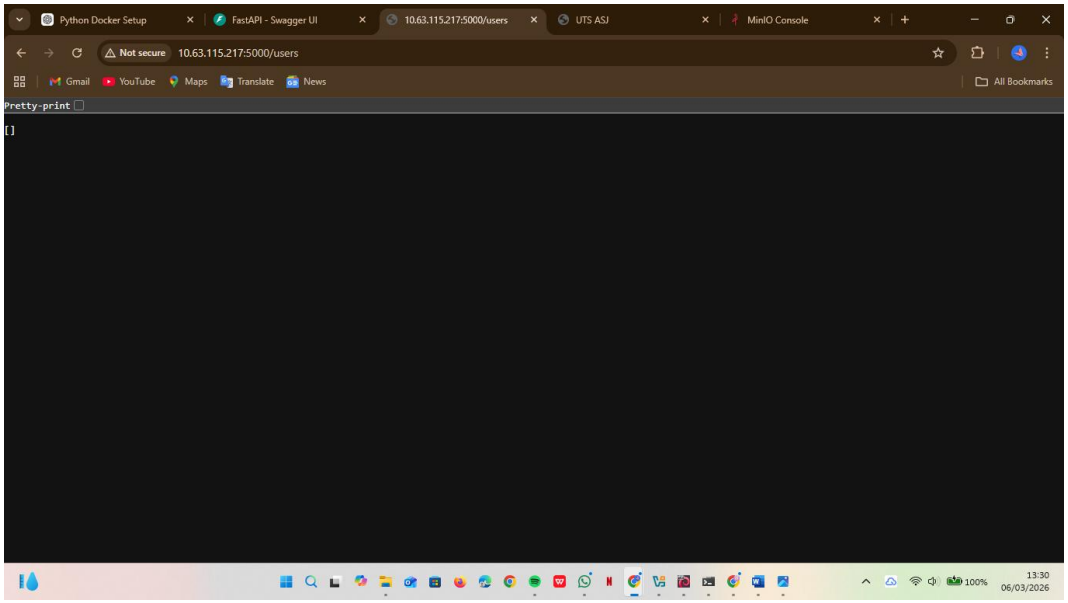
- **FastAPI Documentation**
  Digunakan untuk melihat dan menguji endpoint API yang tersedia pada sistem.

  http://localhost:8000/docs



- **Backend Service**
  Digunakan untuk melihat data user yang diambil melalui backend service.

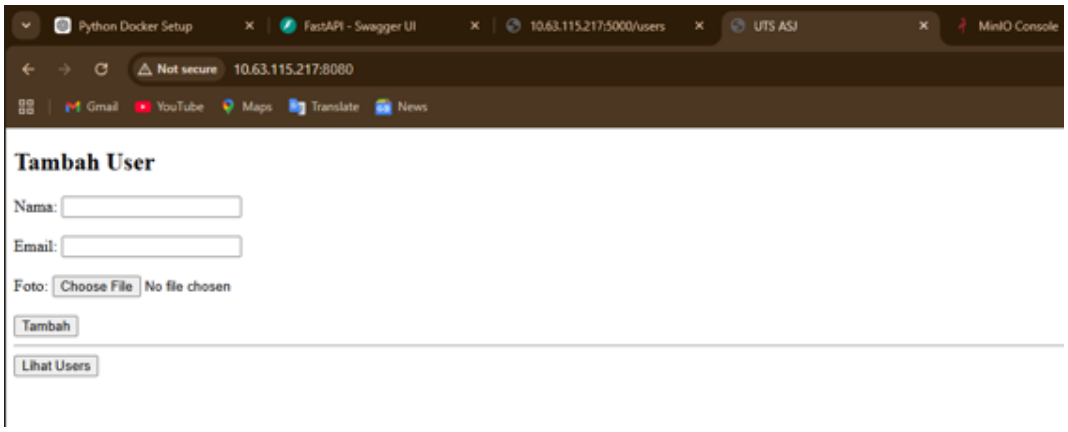
  http://localhost:5000/users



- **Frontend Web**

  Digunakan untuk menampilkan tampilan antarmuka aplikasi yang dapat diakses oleh pengguna.

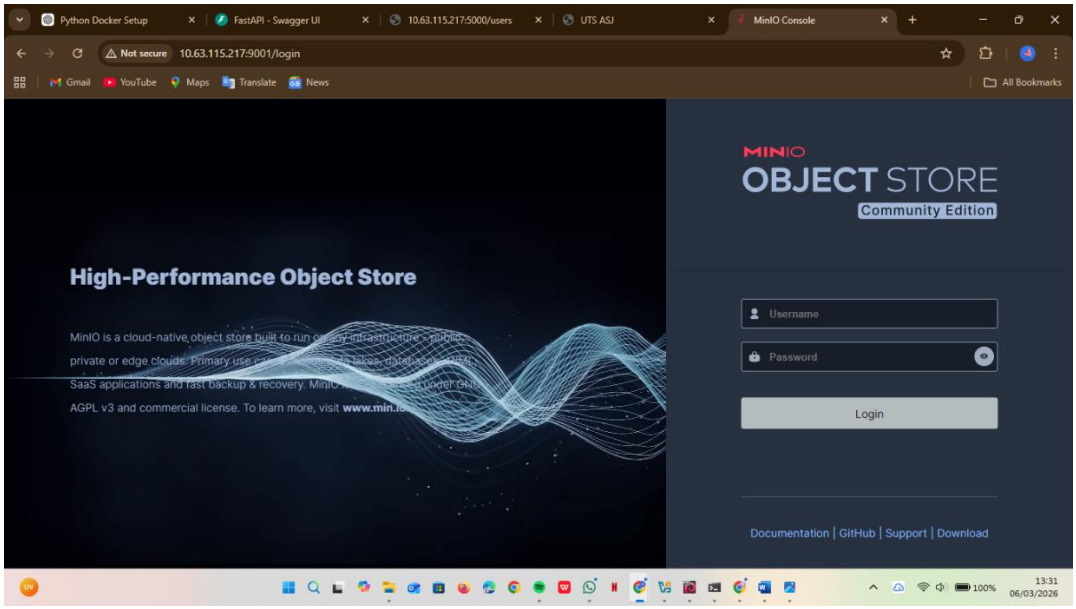
  http://localhost:8080



- **MinIO**

  Digunakan untuk melihat dan mengelola file yang telah diupload ke dalam storage MinIO.

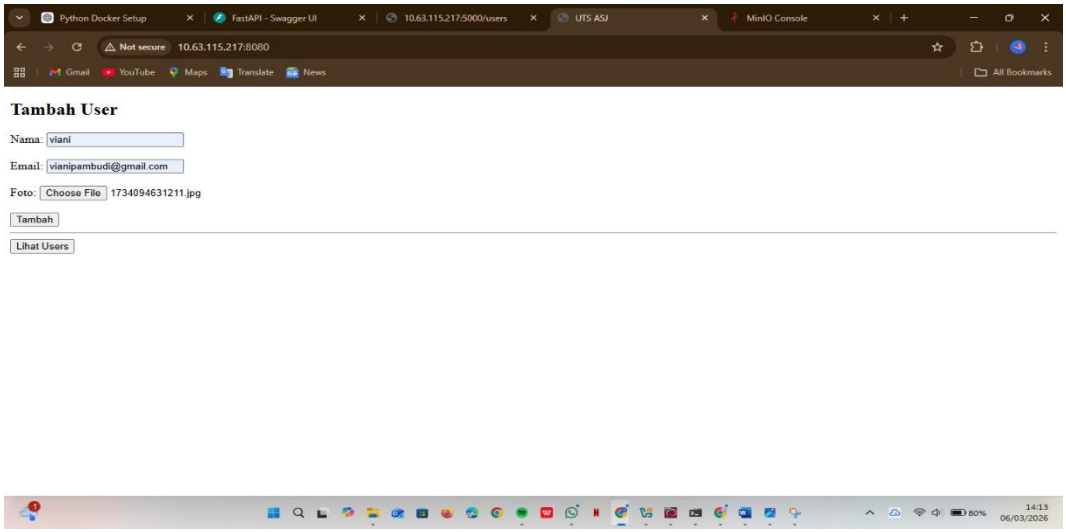
  http://localhost:9001



UJI TESTING

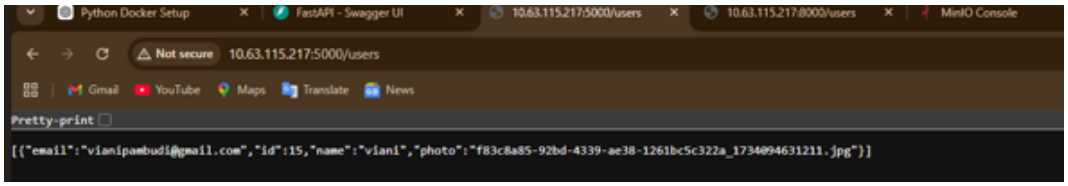
Setelah seluruh service berhasil dijalankan menggunakan Docker Compose, langkah selanjutnya adalah melakukan pengujian terhadap fungsi **CRUD (Create, Read, Update, Delete)** pada sistem.

1. **Create (Menambahkan Data)**
   Proses **Create** dilakukan melalui halaman **frontend** yang dapat diakses melalui web browser. Pada halaman ini pengguna dapat mengisi form data user seperti nama, email, dan foto. Setelah form dikirim, data akan diteruskan ke backend dan kemudian diproses oleh API untuk disimpan ke database.

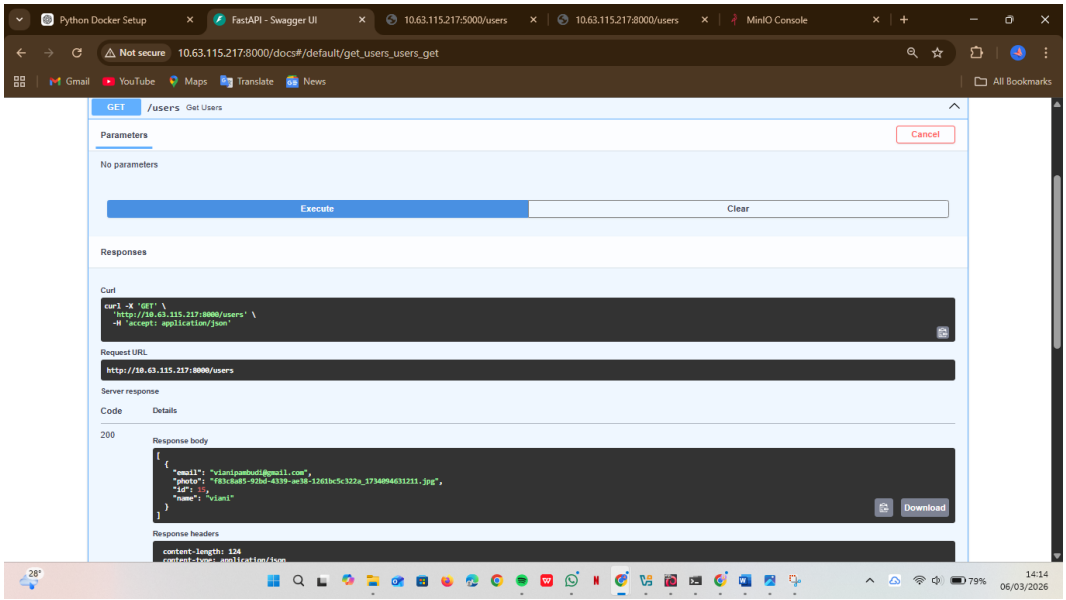


2. Read (Menampilkan Data)

Proses **Read** digunakan untuk menampilkan data user yang telah tersimpan di database. Pengujian ini dilakukan melalui backend service yang mengambil data dari API dan menampilkannya dalam bentuk daftar data user.

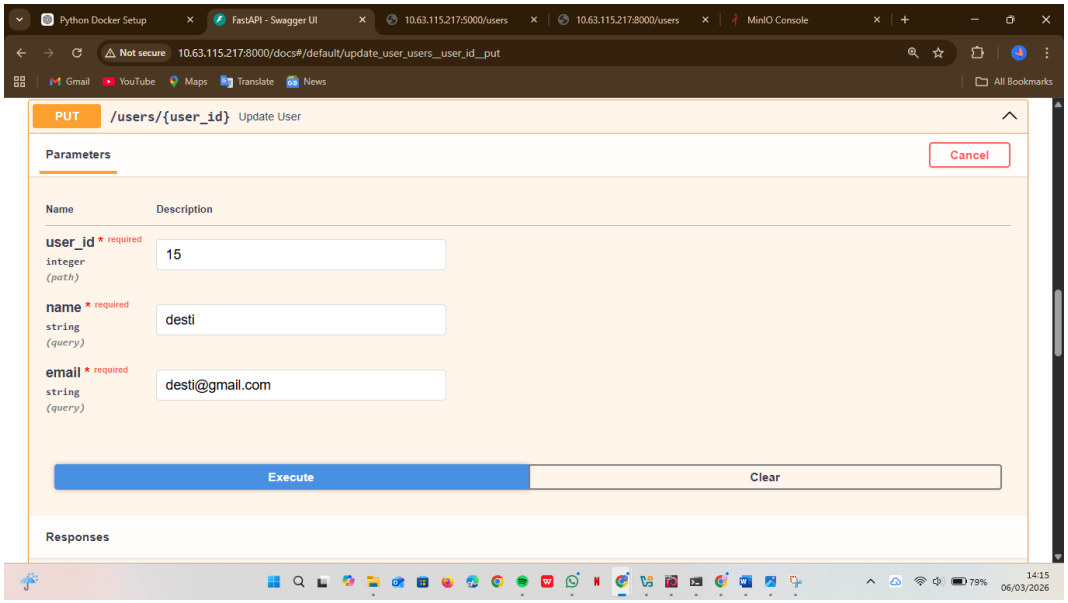


Masuk kedalam FastAPI



3. Update (Memperbarui Data)

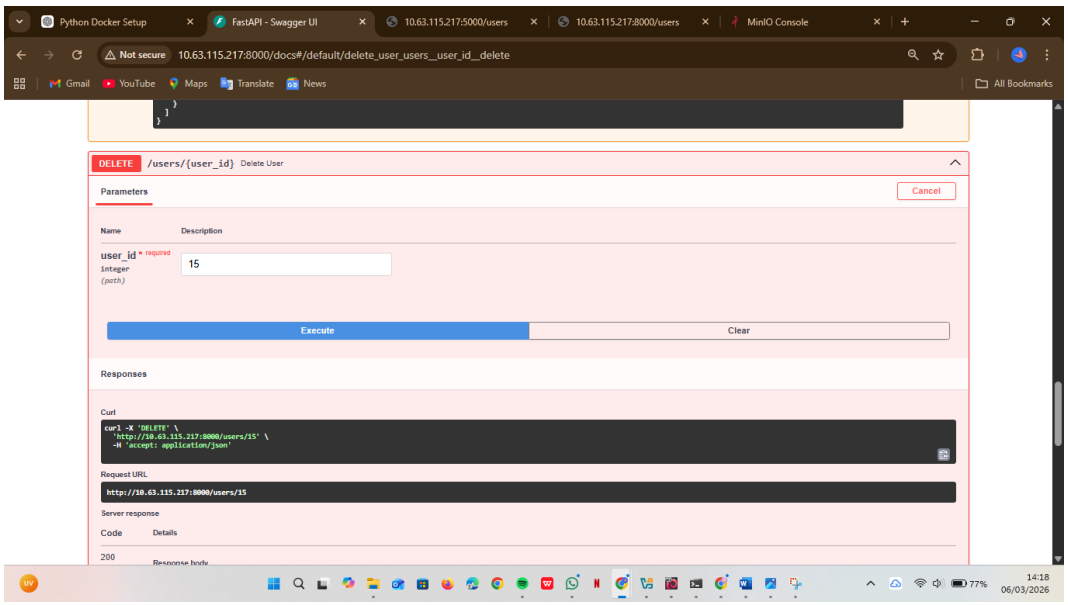
Proses update diuji melalui FastApi Documentation. Pada halaman ini tersedia berbagai endpoint API yang dapat digunakan untuk memperbarui data user yang sudah ada menggunakan fitur *Try it out*.



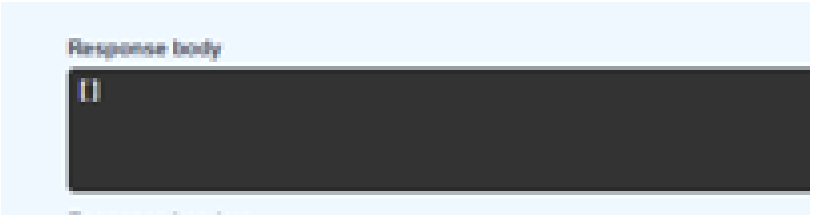
Hasilnya ketika sudah diupdate di FastAPI

4. Delete (Menghapus Data)

Proses Delete diuji melalui FastApi Documentation.



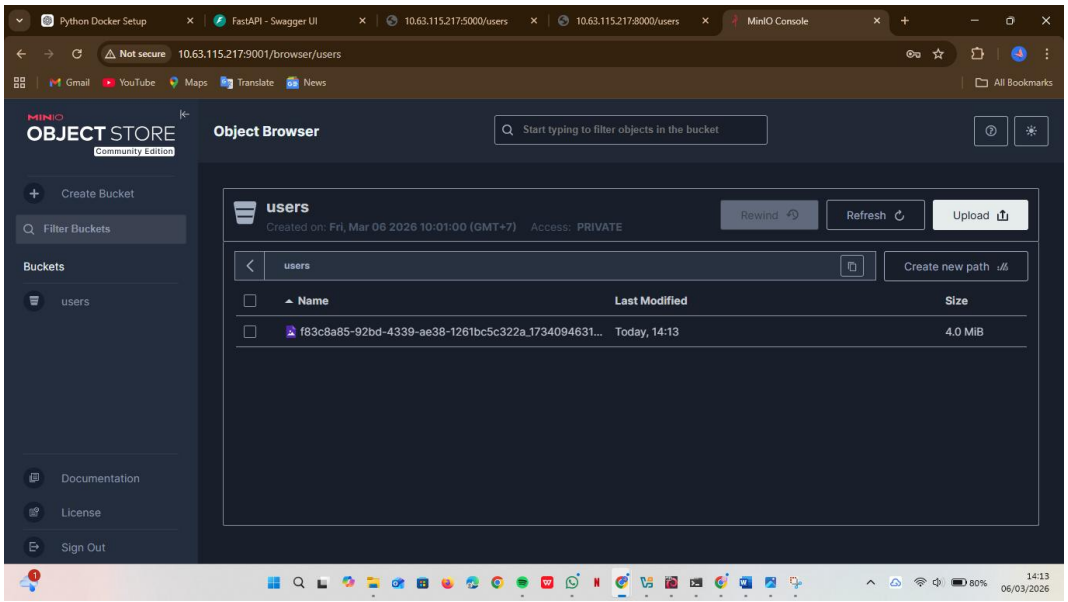
Hasilnya ketika sudah dihapus di FastAPI



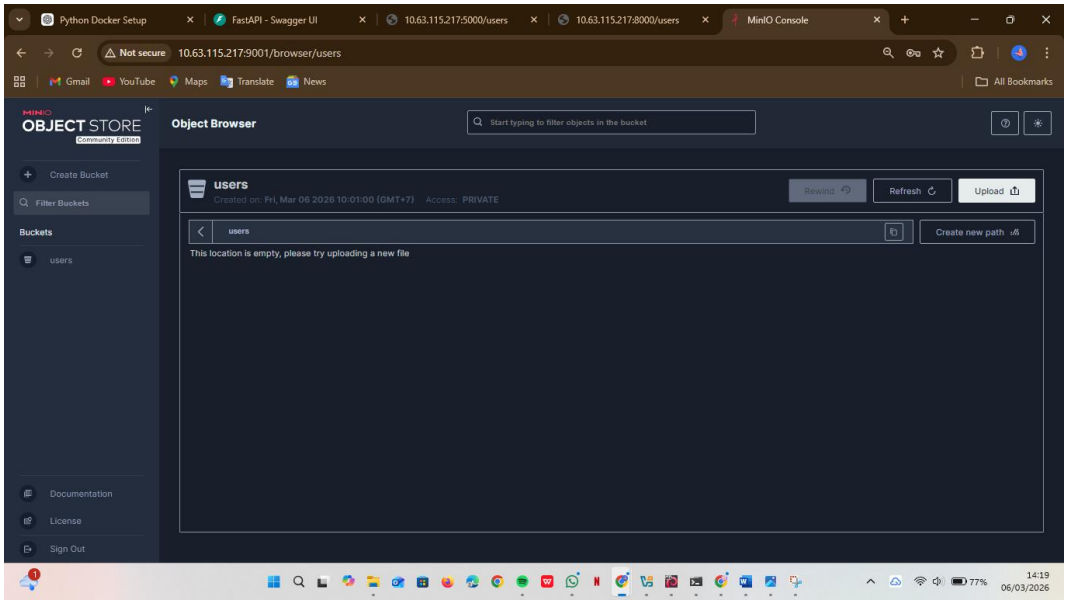
5. Penyimpanan File (MinIO)

Selain data user, sistem juga mendukung proses upload file berupa foto yang disimpan pada layanan **MinIO Object Storage**.

Ketika File Diupload

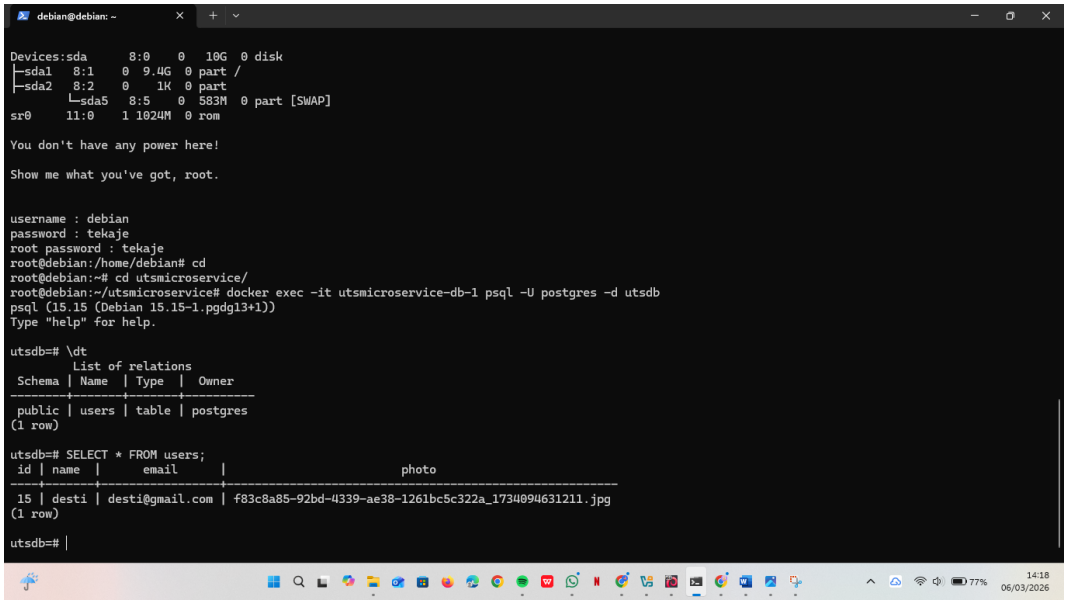


Ketika file dihapus



6. Database PostgreSQL

Database PostgreSQL ketika file di Upload



Database PostgreSQL ketika file di hapus

debian@debian: ~

```
You don't have any power here!

Show me what you've got, root.


username : debian
password : tekaje
root password : tekaje
root@debian:/home/debian# cd
root@debian:~# cd utsmicroservice/
root@debian:~/utsmicroservice# docker exec -it utsmicroservice-db-1 psql -U postgres -d utsdb
psql (15.15 (Debian 15.15-1.pgdg13+1))
Type "help" for help.

utsdb=# \dt
         List of relations
 Schema | Name  | Type  |  Owner
--------+-------+-------+----------
 public | users | table | postgres
(1 row)

utsdb=# SELECT * FROM users;
 id | name  |      email      |                          photo
----+-------+-----------------+-----------------------------------------------------------
 15 | desti | desti@gmail.com | f83c8a85-92bd-4339-ae38-1261bc5c322a_1734094631211.jpg
(1 row)

utsdb=# SELECT * FROM users;
 id | name | email | photo
----+------+-------+-------
(0 rows)

utsdb=# \q
root@debian:~/utsmicroservice#
```

77%
14:19
06/03/2026